

## # Code Metric Visualizer Platform

The docker image for `pmccabe\_visualizer` [https://github.com/SergeyIvanov87/pmccabe\\_visualizer](https://github.com/SergeyIvanov87/pmccabe_visualizer)

The service implementation is still in progress and it has limited functionality.

The service is configured and assembled for different use-cases which are proposed through `docker compose` files. The use-cases would be described in that document later. Only local-machine (Linux tested only) docker images are shipped at the moment.

The service is supposed to monitor/to collect various metrics of code in a C/C++ project. For the moment the only `cyclomatic complexity` is introduced. For more information about `cyclomatic complexity` and how to use it please refer to an appropriate

[article]([https://en.wikipedia.org/wiki/Cyclomatic\\_complexity](https://en.wikipedia.org/wiki/Cyclomatic_complexity)) or for another [repository of mine]([https://github.com/SergeyIvanov87/pmccabe\\_visualizer](https://github.com/SergeyIvanov87/pmccabe_visualizer))

Launching this service by targeting it on your local C/C++ project (or repository), you could leverage API populated by this service to collect, manifest and govern metric subsets.

All communication with the particular `microservice`/docker container is available either through simplest pseudo-filesystem API, which is populated in [API manifest](cyclomatic\_complexity/API.fs), or using HTTP queries if you decided to employ [REST API](rest\_api).

Whatever you preferred, the API subset remains the same. Thanks to the REST ideology, it is possible to generate both sets of API: requests could represent a particular hierarchy structure, thus

`pmccabe\_collector` leverages this idea and maps those API requests as a structure of nodes mapped to a filesystem hierarchy as directories and files inside the populated API-entry point

`api.pmccabe\_collector.restapi.org` resided in your project directory.

Each request can be executed as simple ACCESS-operation on a file named `exec` or

`modify\_this\_file` in the bottom of relevant filesystem hierarchy in the same way as the Linux `/proc` pseudo-filesystem employed in order to read (and/or store) some system settings. For more information how to use psedo-filesystem API please follow this document [How-To-Use-API](HowTo/How-To-Use-API.md)